

Van Core tot Cloud: De Reis van een Modern IoT-Apparaat



Een Gids voor het Bouwen van Geconnecteerde Embedded Systemen met Moderne Tools.

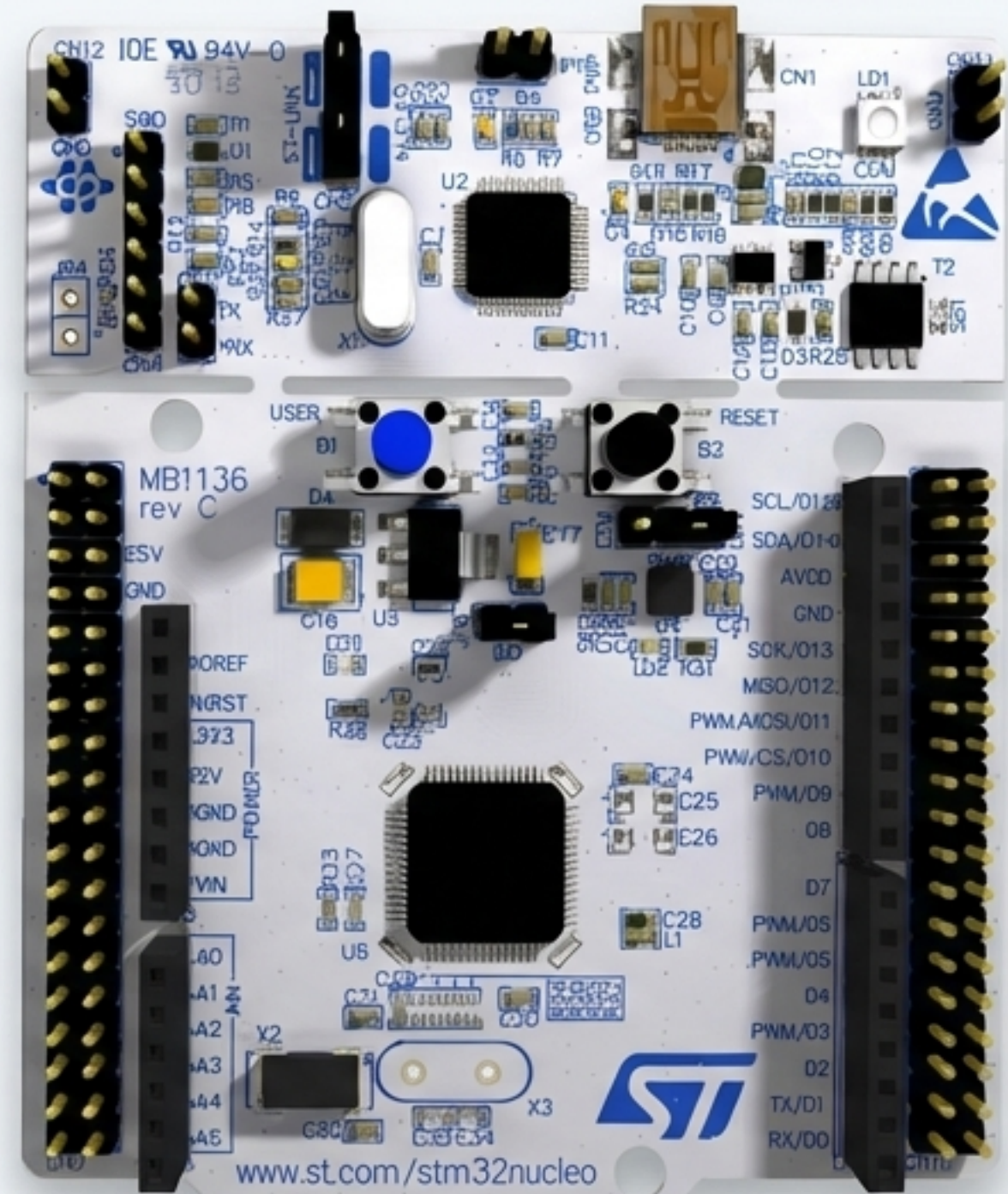
Uw Reis Begint Hier: Bouwen op een Solide Basis

Wat u al kent

- Werken met de NUCLEO-091RC.
- Programmeren in C.
- Basis I/O, UART, SPI, I²C.
- Register-level controle met CMSIS-bibliotheken.

Onze Nieuwe Toolkit

- **De Hardware:** NUCLEO-L476RG, een krachtigere microcontroller met kloksnelheden tot 80MHz en extra interfaces zoals USB.
- **De Software:** De Hardware Abstraction Layer (HAL)-bibliotheken, een modernere aanpak.
- **De Doelstelling:** Complexe systemen bouwen met WiFi-communicatie en een Realtime Operating System (RTOS).



De Eerste Stap: De Hardware Abstraheren met HAL

We stappen af van directe registermanipulatie (CMSIS) naar een hogere abstractielaag (HAL).

CMSIS (Cortex Microcontroller Software Interface Standard)	HAL (Hardware Abstraction Layer)
Voordeel: Efficiënt, controle tot op bit-niveau. Nadeel: Niet eenvoudig overdraagbaar naar een andere microcontroller.	Voordeel: Maakt code onafhankelijker van de hardware, wat overstappen naar een andere MCU vereenvoudigt. Ligt als een softwarelaag bovenop CMSIS. Nadeel: Kan potentieel minder efficiënt zijn.



CMSIS	HAL
<pre>// Clock voor GPIOA inschakelen. RCC->AHB2ENR = RCC->AHB2ENR RCC_AHB2ENR_GPIOAEN;</pre>	<pre>HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);</pre>

Een Moderne Workflow: Visuele Configuratie met STM32CubeMX

Met HAL kunnen we de microcontroller grafisch configureren in plaats van handmatig registers in te stellen. Dit versnelt de ontwikkeling aanzienlijk.

1. Selecteer je Board

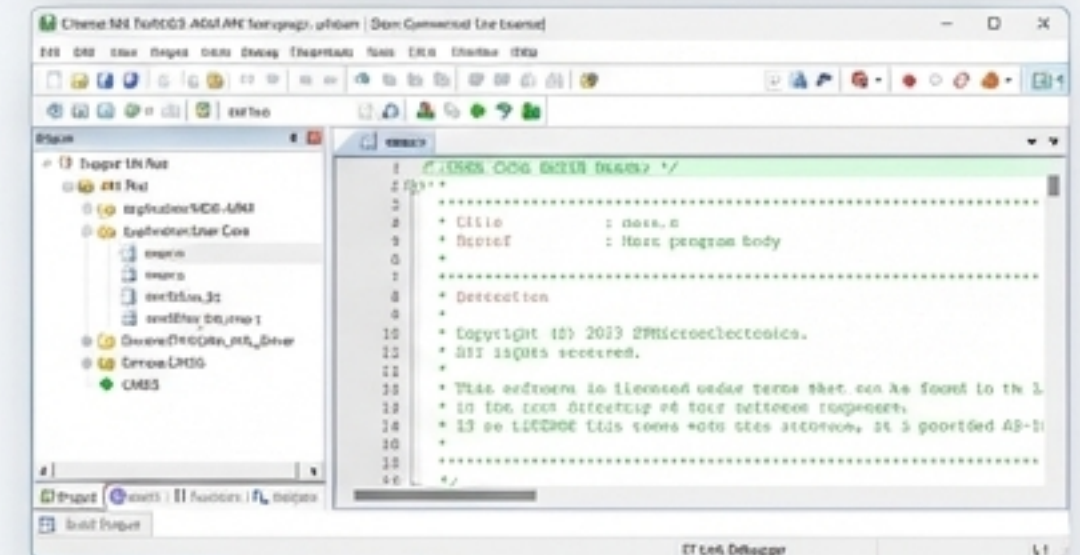
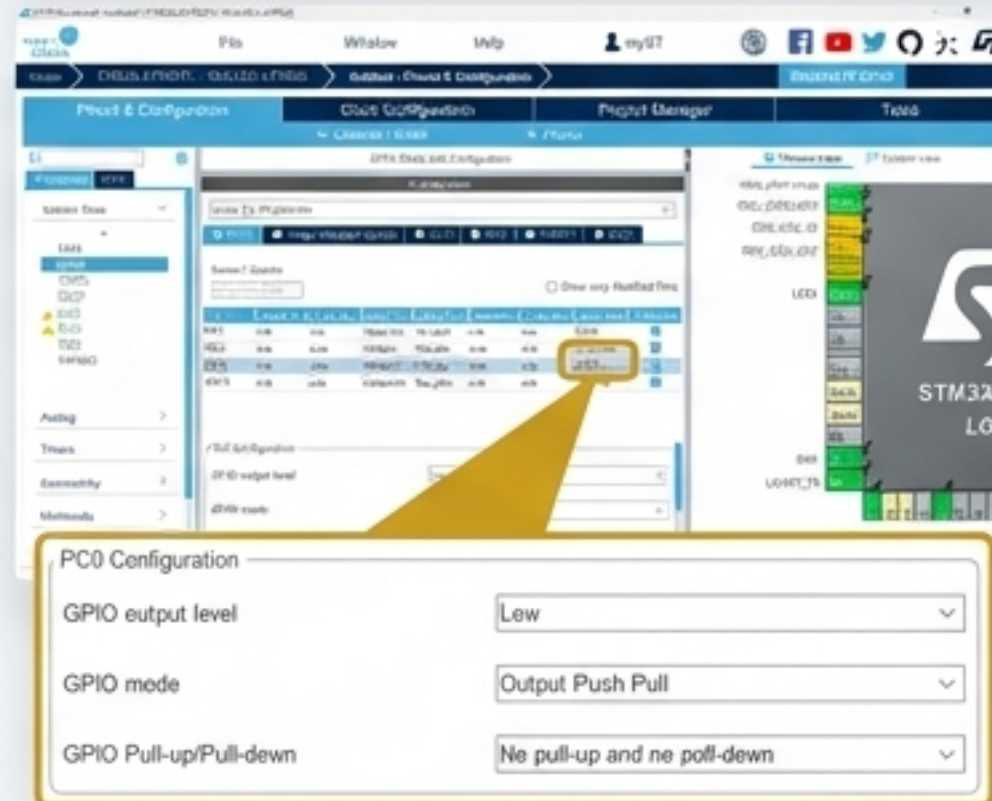
Kies de NUCLEO-L476RG in de Board Selector.

2. Configureer Pinnen

Wijs functies toe aan pinnen (bv. GPIO Output voor een LED) in de Pinout & Configuration view.

3. Genereer Code

CubeMX creëert een volledig Keil **µVision**-project met alle initialisatiecode.



De Zintuigen Uitbreiden: Lokale Communicatie via SPI

Wat is SPI?

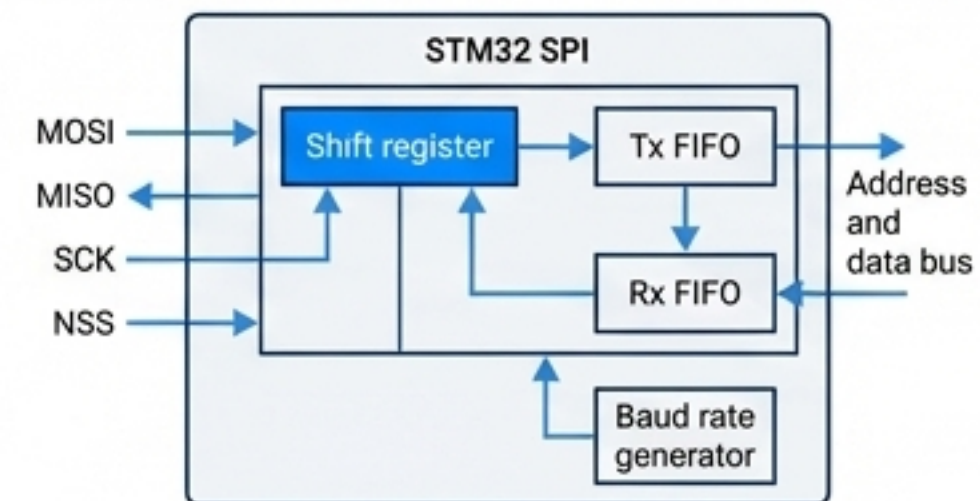
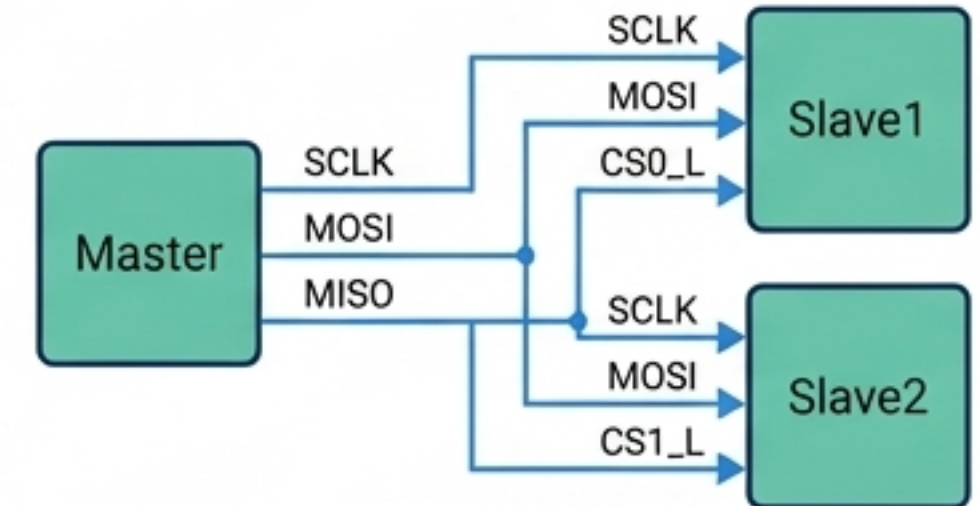
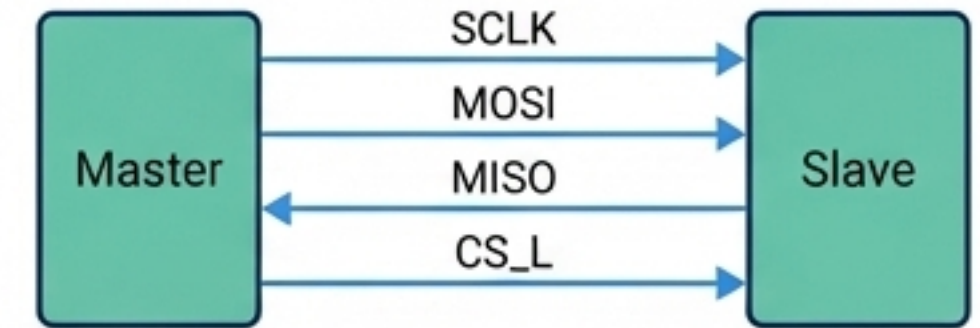
- Serial Peripheral Interface: een synchrone communicatiebus.
- In tegenstelling tot UART (asynchroon), gebruikt SPI een kloksignaal (SCK) en is er geen afgesproken baudrate nodig.
- Ideaal voor snelle communicatie tussen IC's op dezelfde printplaat.

De Sleutel Signalen

- **MOSI**: Master Output Slave Input
- **MISO**: Master Input Slave Output
- **SCK**: Serial Clock
- **(N)SS / CS**: Slave Select / Chip Select

Topologieën

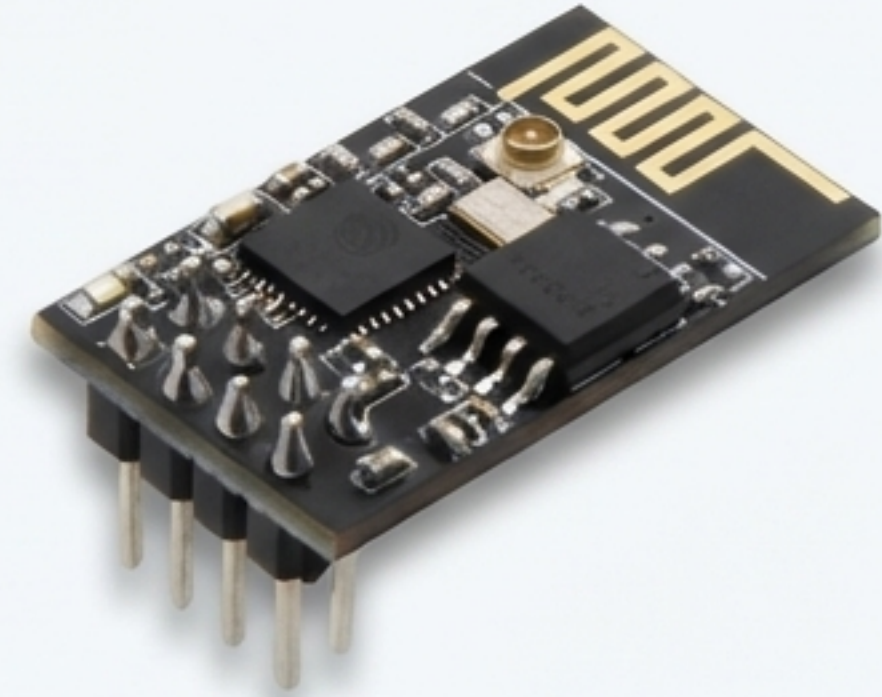
- Point-to-Point
- Star
- Daisy Chain



De Sprong naar het Internet: Connectiviteit via WiFi

De Hardware

De LILYGO T-01C3 module met een Espressif ESP32-C3 chip. Dit is onze brug naar het internet.

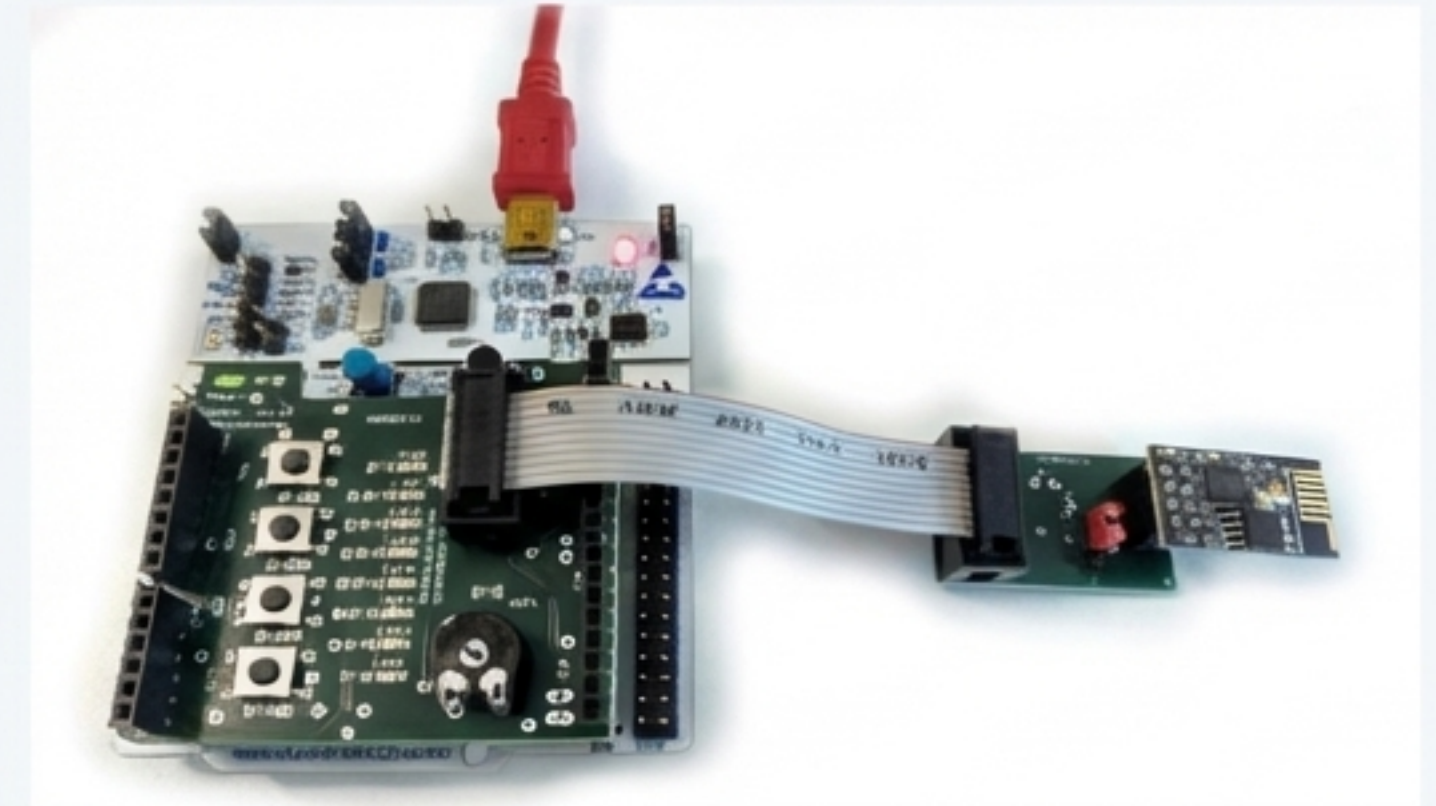


Hoe het werkt

De Nucleo communiceert met de ESP32-module via AT-commando's over een seriële poort (UART). De ESP32 handelt de complexiteit van het WiFi-protocol af.

Het Conceptuele Kader: De TCP/IP Stack

7. Applicatie	HTTP, FTP, DNS, RTP
4. Transport	bijv. TCP, UDP, SCTP
3. Netwerk	TCP/IP
2. Data Link	Ethernet, Token Ring
1. Fysiek	Fysieke media



De Taal van het Web: Data Uitwisselen met HTTP & API's

Kernprotocollen

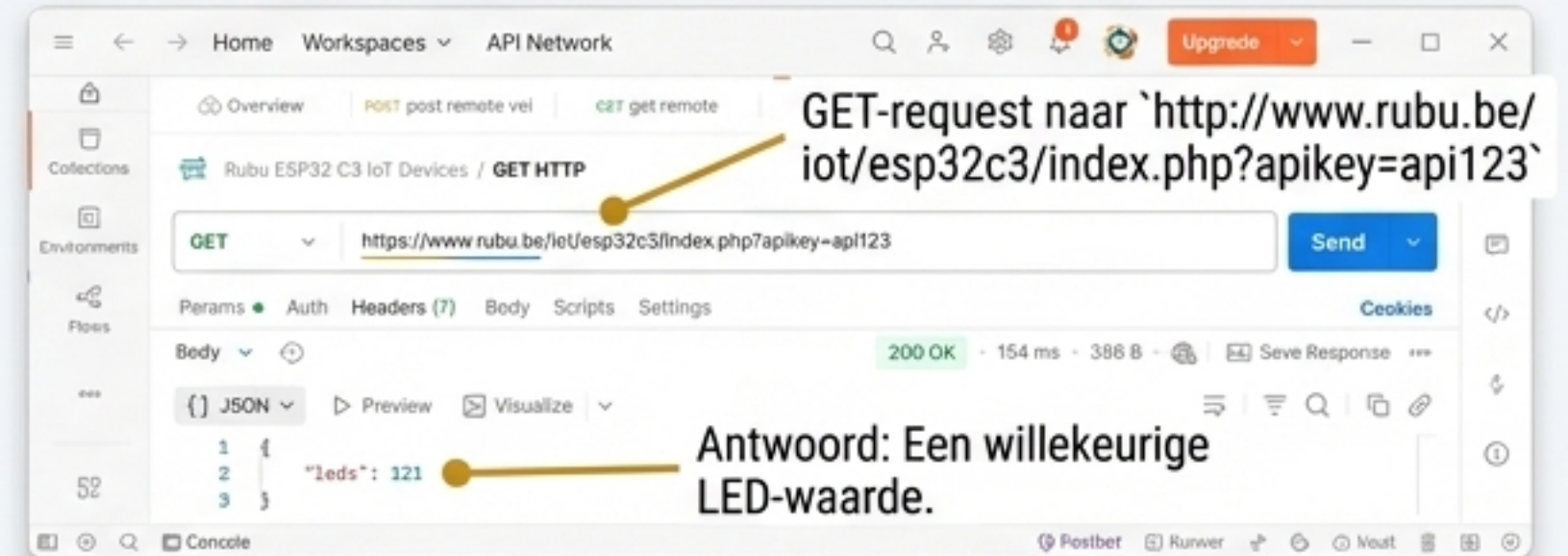
Roboto Flex:

- **HTTP GET:** Data opvragen van een server (bv. de huidige status van een apparaat).
- **HTTP POST:** Data naar een server sturen (bv. een nieuwe sensorwaarde uploaden).

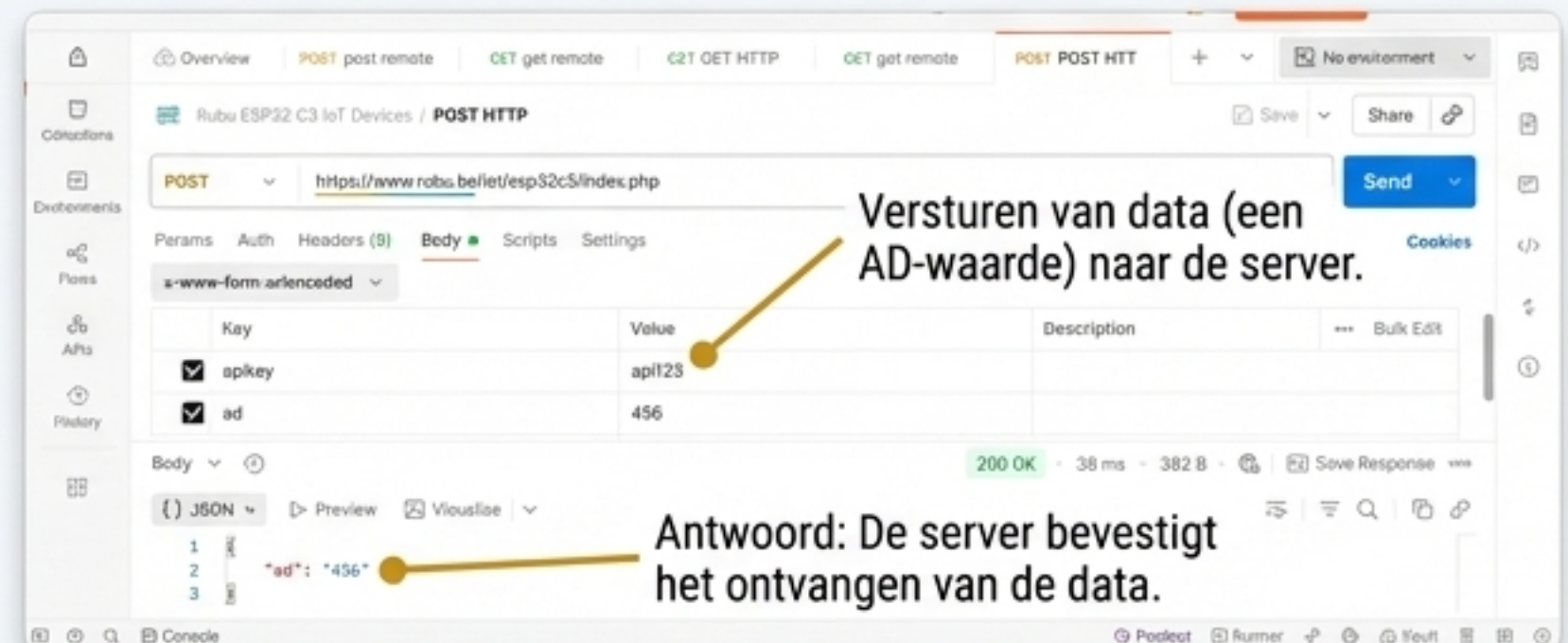
Testen als een Pro: Postman

Postman is een API-platform om dataverzoeken te bouwen en te testen. Hiermee kunnen we onze web API testen voordat we deze in de microcontroller implementeren.

Praktijkvoorbeeld: GET-request



Praktijkvoorbeeld: POST-request



Een Efficiënter Gesprek: MQTT voor IoT

Waarom MQTT?

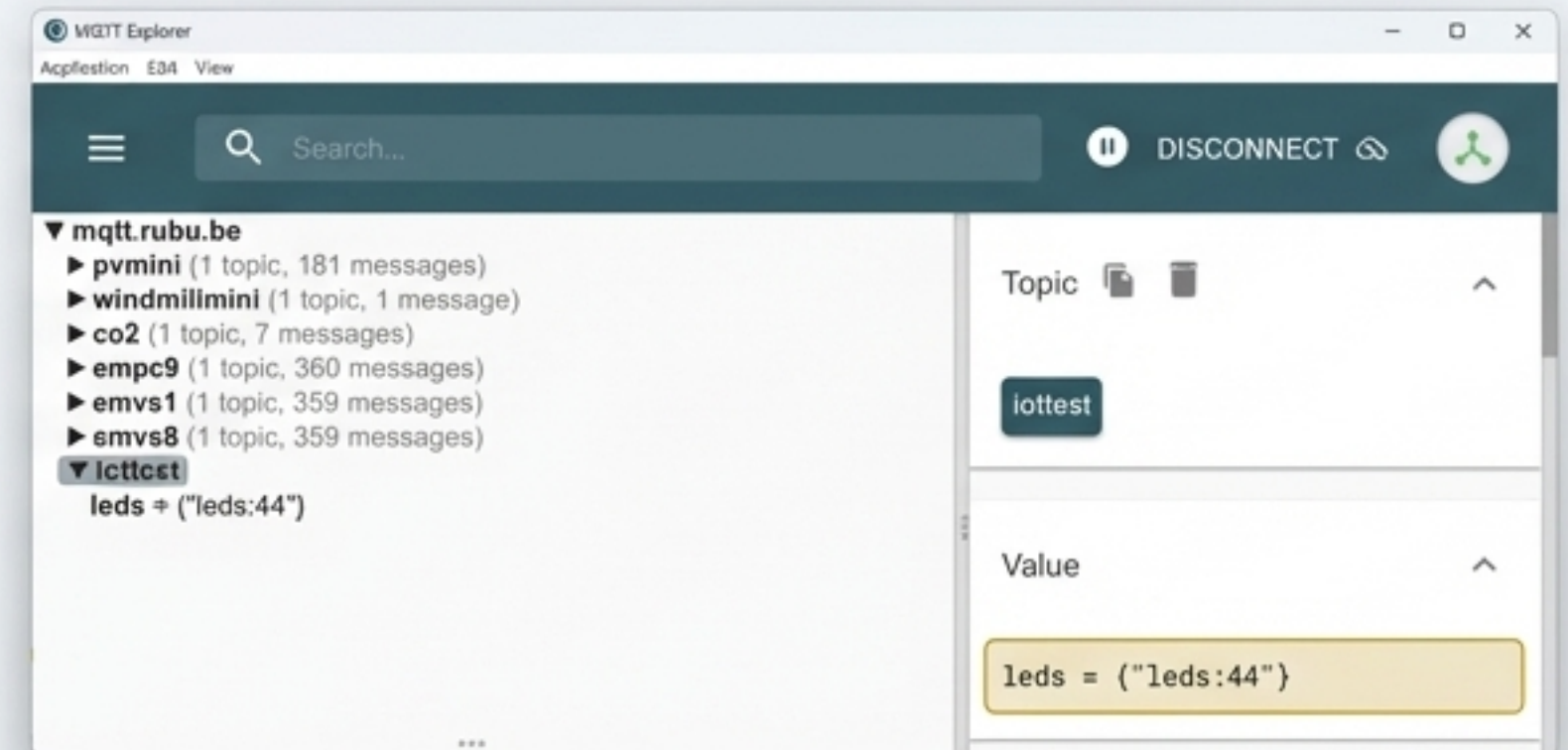
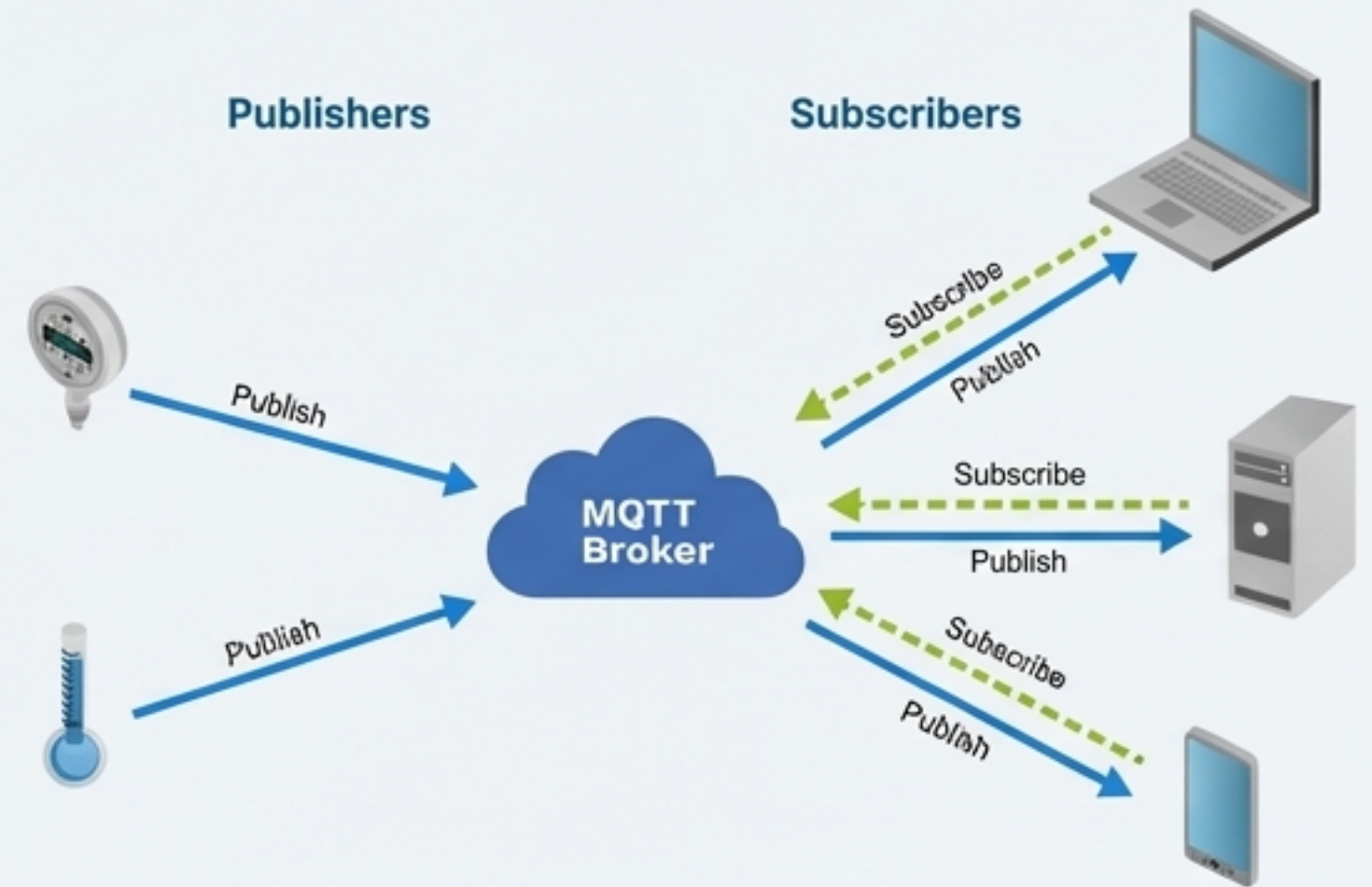
HTTP werkt volgens een request-response model. MQTT (Message Queuing Telemetry Transport) is een lichtgewicht publish-subscribe protocol, veel efficiënter voor IoT-toepassingen.

Hoe het werkt: Publish-Subscribe

- Publishers (bv. sensoren) sturen berichten naar een 'topic' op een centrale server.
- Subscribers (bv. een app of een ander apparaat) luisteren naar dat 'topic'.
- De MQTT Broker is de centrale hub die de berichten routeert.

De Implementatie

Onze ESP32-module kan met de juiste AT-commando's MQTT-berichten publiceren en zich abonneren op topics.



De Uitdaging: De Grenzen van de 'Superloop'

De Situatie

-  Knoppen uitlezen
-  Een sensorwaarde meten (bv. via SPI)
-  Periodiek data publiceren via MQTT
-  Reageren op commando's die via HTTP binnenkomen
-  LEDs aansturen om status te tonen



Het Probleem

- HAL_Delay(500) bevriest de hele applicatie voor een halve seconde.
- Wachten op een HTTP-antwoord kan seconden duren, waarin geen knoppen kunnen worden gelezen.
- Complexe logica wordt snel een onbeheerbare 'spaghetti' van code.

De 'superloop' is niet schaalbaar voor een modern, geconnecteerd apparaat.
We hebben een dirigent nodig die het orkest kan leiden.

De Oplossing: Een Dirigent voor uw Systeem - Het RTOS

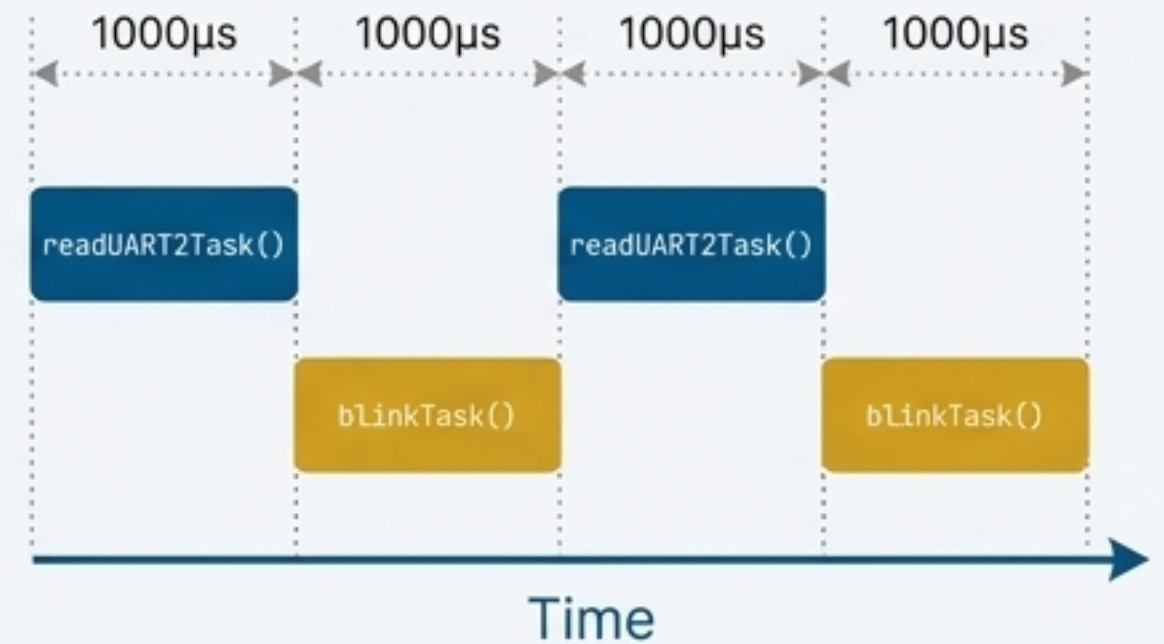
Een 'preemptive multitasking operating system'.

Kernconcepten

- **Tasks (of Threads):** Onafhankelijke C-functies die elk een specifieke taak uitvoeren (bv. `bLinkTask`, `readUART2Task`).
- **Scheduler:** Het brein van het RTOS. De scheduler bepaalt welke taak op welk moment mag draaien.
- **Preemptive Multitasking:** De scheduler kan een taak onderbreken om een taak met hogere prioriteit voorrang te geven. Dit geeft de illusie dat taken parallel worden uitgevoerd.

Voordelen

- Maakt complexe software modulair en beheersbaar.
- Taken kunnen wachten zonder de hele CPU te blokkeren.
- Verbetert de responsiviteit van het systeem.



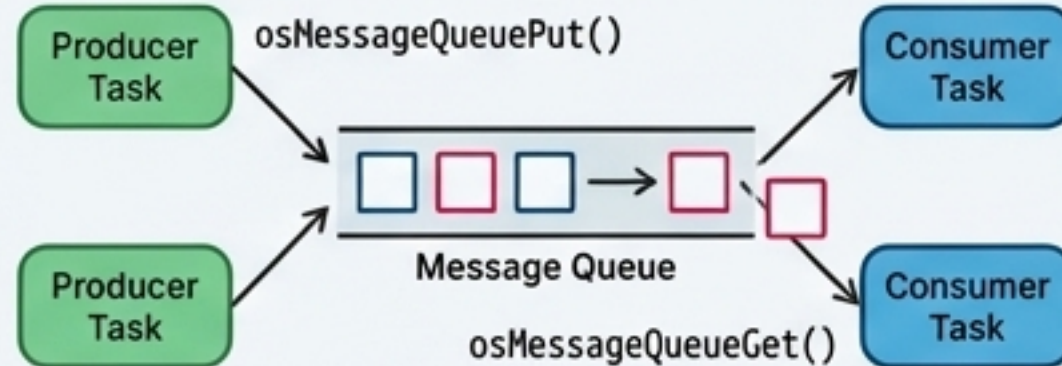
De Gereedschapskist van het RTOS: Samenwerking Tussen Taken

De Uitdaging: Hoe kunnen taken veilig communiceren en gedeelde bronnen (zoals de UART of een globale variabele) gebruiken?

De Oplossing: Synchronization Primitives.

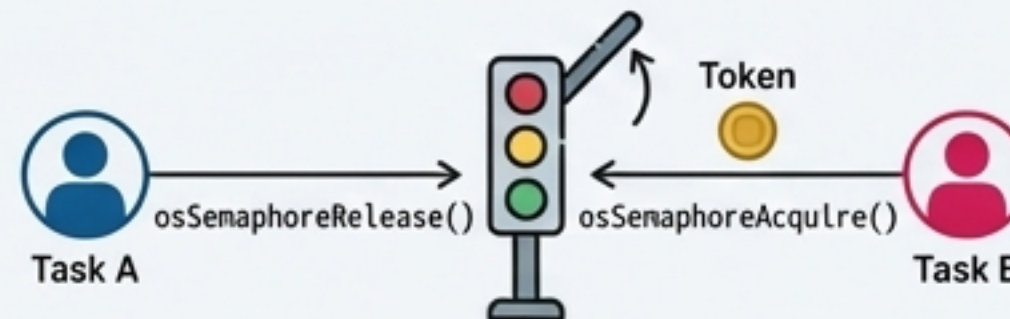
Message Queue

Een FIFO-wachtrij om data veilig van de ene taak naar de andere te sturen. Een 'producer' taak voegt items toe, een 'consumer' taak haalt ze eruit.



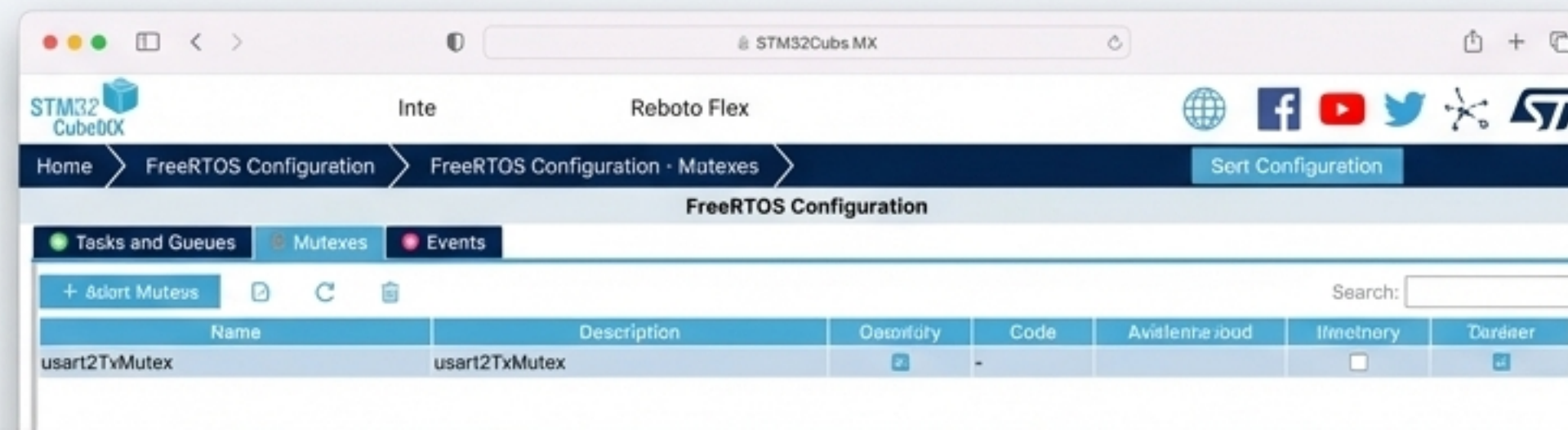
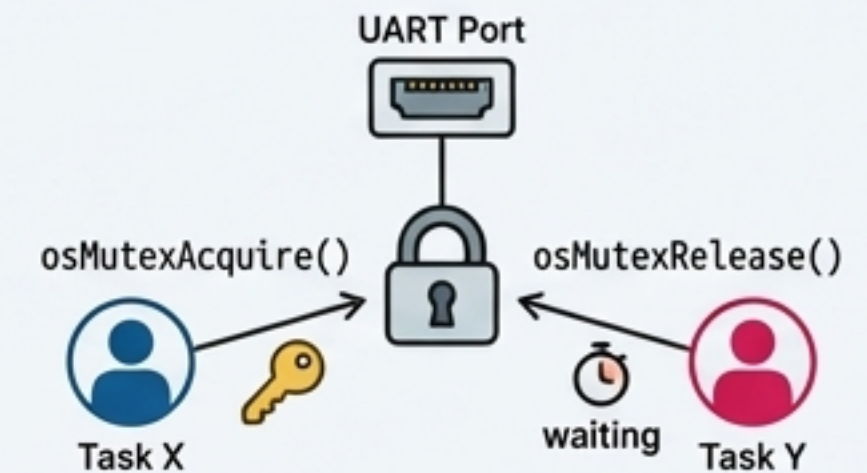
Semaphore

Een signaalmechanisme. Een taak kan wachten op een 'token' (acquire) dat een andere taak vrijgeeft (release). Handig om de ene taak op de andere te laten wachten.



Mutex (Mutual Exclusion)

Een 'sleutel' om exclusieve toegang tot een gedeelde bron te garanderen. Slechts één taak kan de mutex tegelijk bezitten, wat conflicten voorkomt.



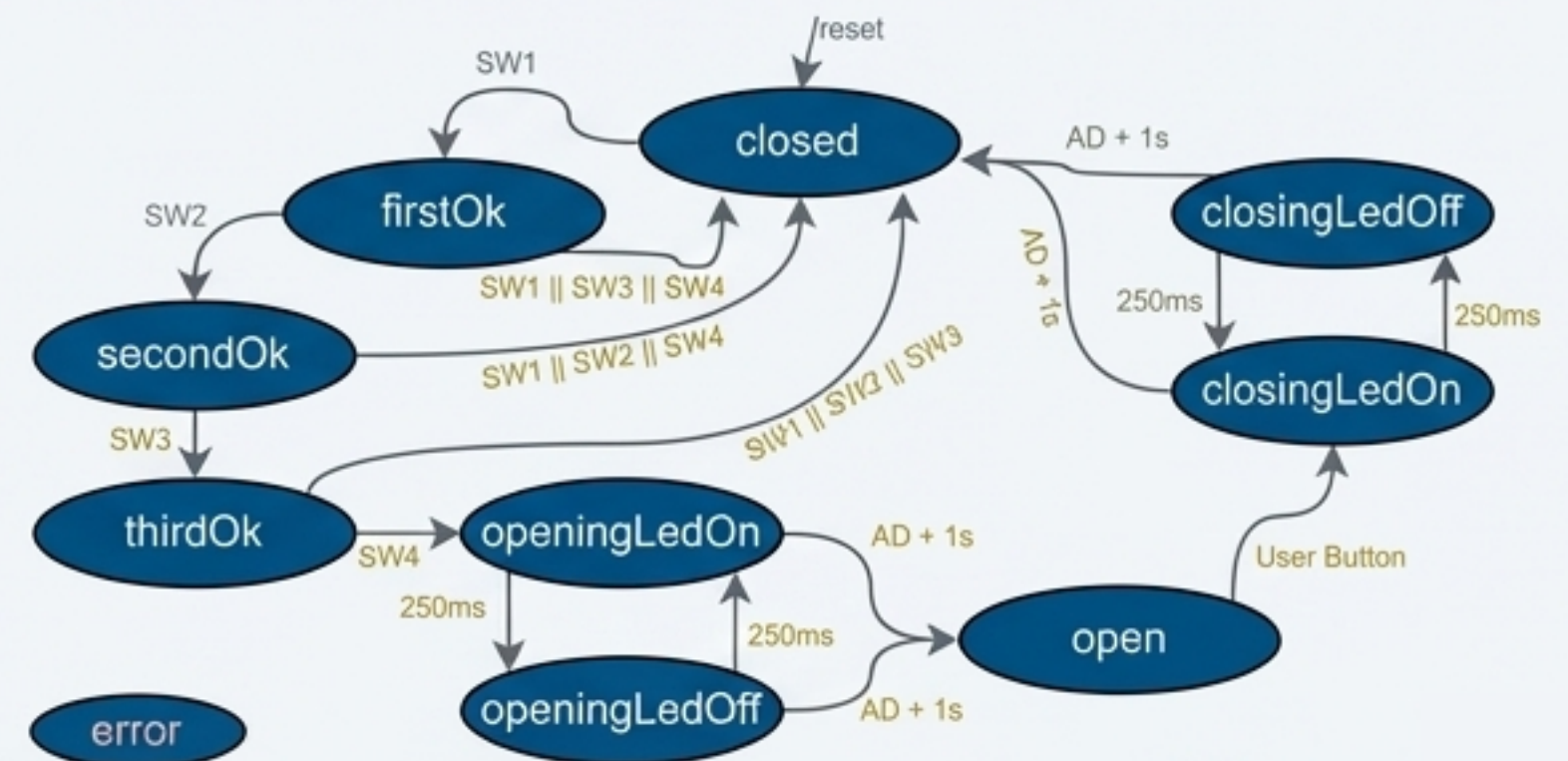
Capstone Project: De IoT Toegangspoort

Een op afstand bestuurbare toegangspoort. De status van de poort (open/gesloten) wordt gevisualiseerd op een webpagina en kan via de cloud worden bediend.

De Architectuur - Een Systeem van Taken

De complexiteit wordt beheerd door het op te splitsen in gespecialiseerde RTOS-taken:

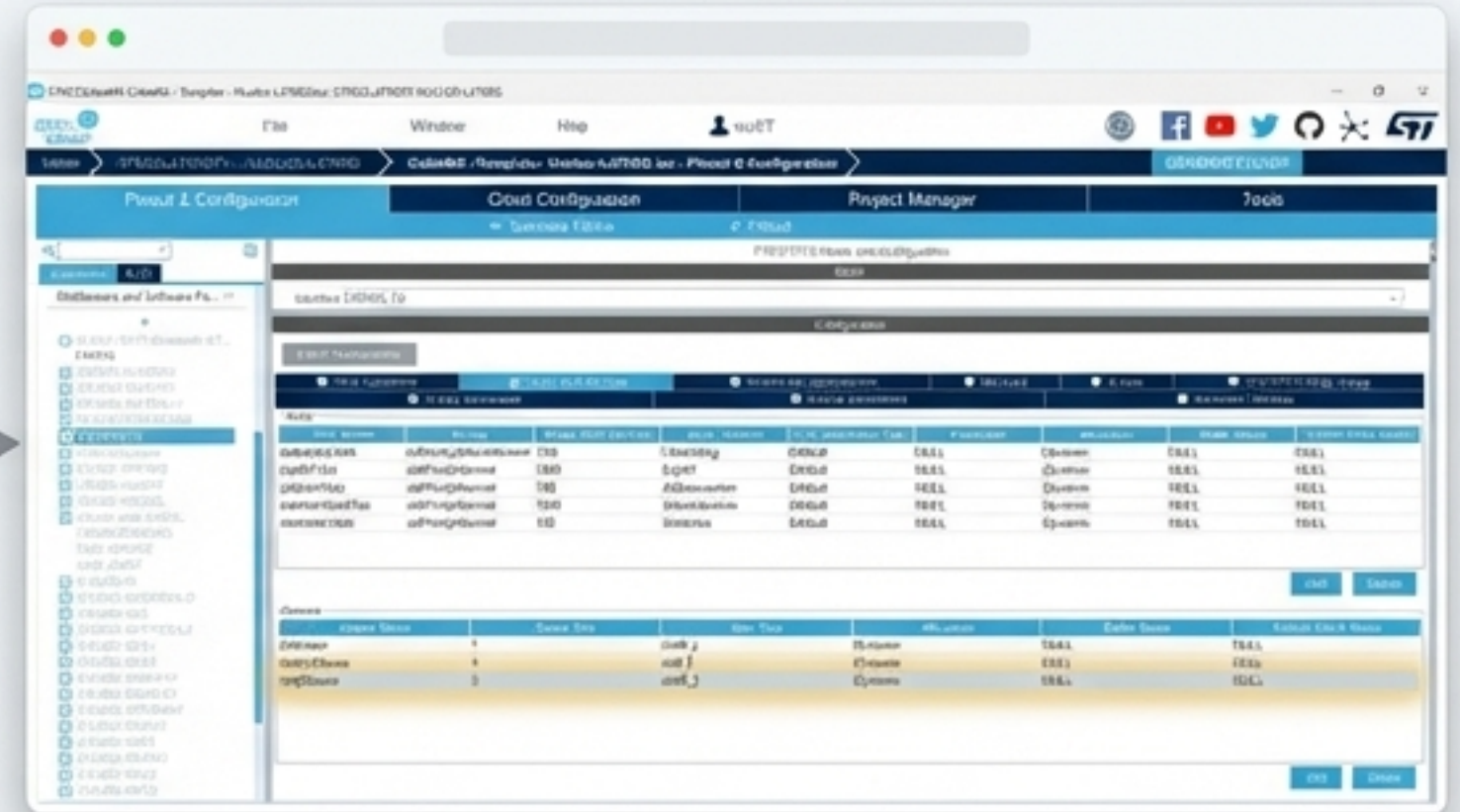
- watchdogTask: Houdt het systeem levend.
- esp32Task: Beheert alle communicatie met de WiFi-module.
- adConvTask: Leest een analoge waarde (bv. voor poortsnelheid).
- switchesTask: Leest lokale bedieningsknoppen.
- stateMachineTask: Het "brein" dat de logica van de poort beheert (openen, sluiten, foutstatus).



De Toegangspoort Onder de Motorkap: RTOS in Actie

Praktische Toepassingen van Primitives

- **Queue (`mqttQueue`):** De `stateMachineTask` plaatst statusupdates (bv. 'open') in een queue. De `esp32Task` leest deze queue uit en publiceert de updates via MQTT. Dit ontkoppelt de systeemlogica van de communicatielogica.
- **Mutex (`usartMutex`):** Zowel de `esp32Task` (voor AT-commando's) als andere debug-taken willen mogelijk naar de UART schrijven. Een mutex garandeert dat hun output niet door elkaar loopt.
- **Event Flags:** Gebruikt om te signaleren tussen taken, bv. de `switchesTask` zet een vlag wanneer een knop wordt ingedrukt, waar de `stateMachineTask` op wacht.



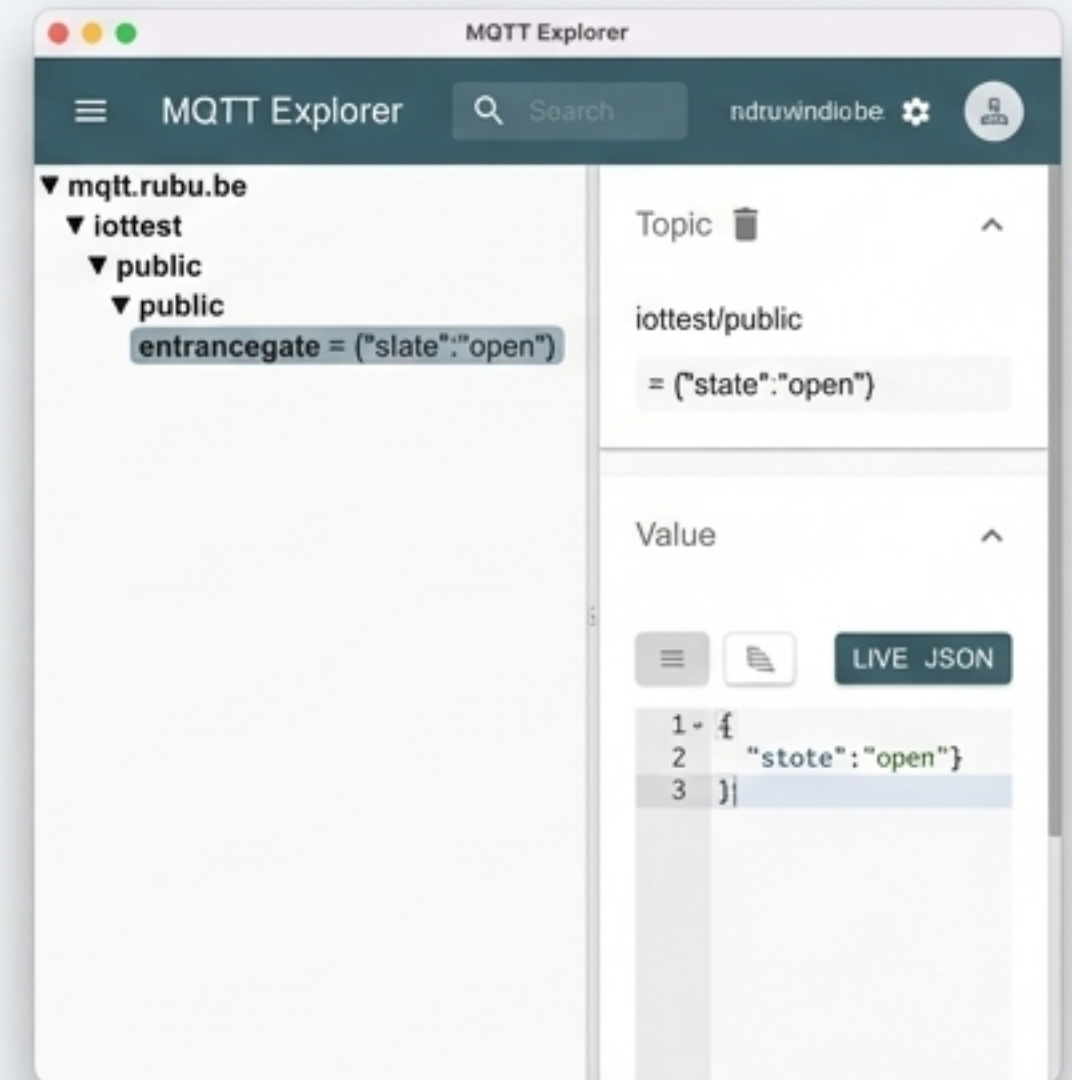
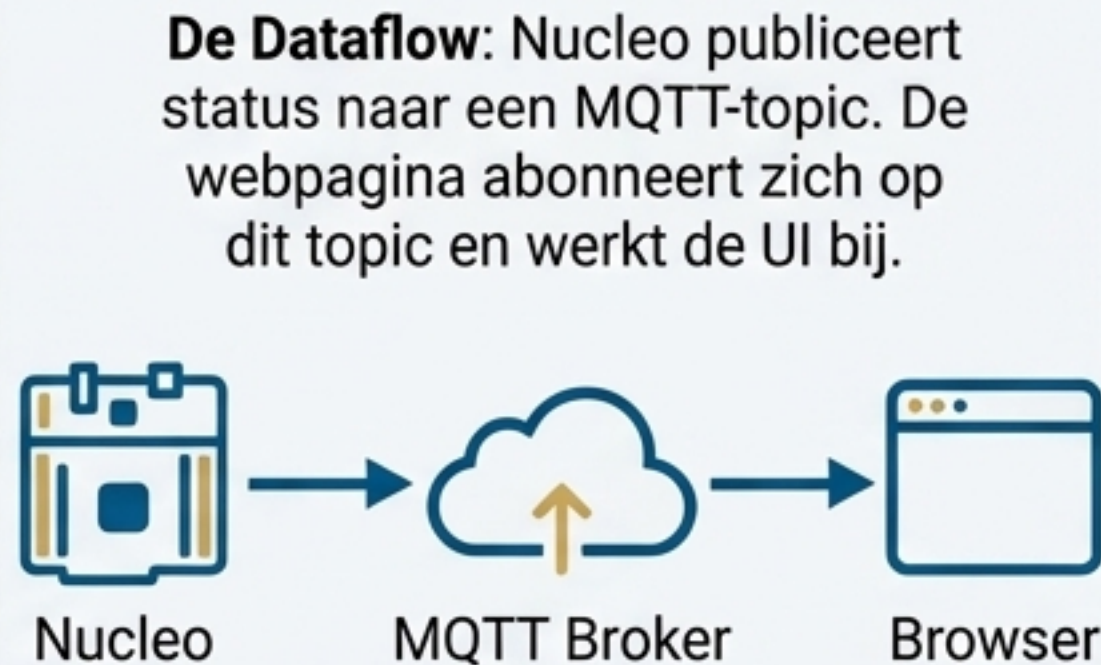
```
/* Infinite loop */
for(;;)
{
    // Is er een nieuwe state opgehaald?
    osStatus = osMessageQueueGet(mqttQueueHandle, &state, NULL, 1);
    if(osStatus == osOK)
    {
        if(state == open)
        {
            // MQTT-bericht publish'n.
            MqttPublish(MQTT_PUB_TOPIC, "{\"state\":\"open\"}");
        }

        if(state == closed)
        {
            // MQTT-bericht publish'n.
            MqttPublish(MQTT_PUB_TOPIC, "{\"state\":\"closed\"}");
        }
    }
}
```

Het Resultaat: Een Fysiek Apparaat Verbonden met de Cloud



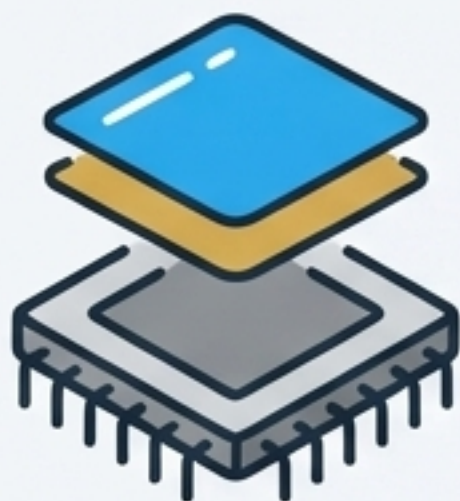
De **Gebruikersinterface**: een webpagina die de actuele status van de poort toont.



Het Bewijs: Live databerichten die binnenkomen in MQTT Explorer.

Uw Nieuwe IoT Superkrachten

U heeft de fundamentele bouwstenen geleerd om elk modern, geconnecteerd apparaat te ontwerpen en te bouwen.



Hardware Abstraction

Efficiënt en overdraagbaar programmeren met **HAL** en **STM32CubeMX**.



Lokale Communicatie

Peripherals zoals sensoren en actuators aansturen via protocollen als **SPI**.



Cloud Connectiviteit

Apparaten met het internet verbinden via **WiFi**, en data uitwisselen met webdiensten via **HTTP** en **MQTT**.



Systeem Architectuur

Complexe, gelijktijdige taken beheren met de kracht en structuur van een **Real-Time Operating System (RTOS)**.